

```

import java.io.*;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class GradeAnalyzer {

    private static List<String> invalidScores = new ArrayList<String>();
    private static Integer maxScore = Integer.MIN_VALUE;
    private static Integer minScore = Integer.MAX_VALUE;
    private static Map<String, Integer> gradeCounter = new HashMap<String, Integer>(
        Map.of("A", 0, "B", 0, "C", 0, "D", 0, "F", 0));

    // Main Method to execute the code
    public static void main(String[] args) {
        String fileName = "scores.txt"; // fall back to scores.txt
        String outputFileName = "result.txt";
        if (args.length == 0) {
            System.out.println("No arguments were provided.");
            System.out.println("Looking for default scores.txt and generate
report.");
        } else {
            String firstArg = args[0];
            if (firstArg == null || firstArg.isEmpty()) {
                System.out.println("No Argument provided");
            } else {
                fileName = firstArg;
                System.out.println("Parsing the provided file" + fileName + " and
generate report.");
            }
        }

        ArrayList<Integer> scores = readScores(fileName);
        analyzeScores(scores);
        writeReport(scores, calculateAverage(scores), maxScore, minScore,
invalidScores, outputFileName);
    }

    // method that returns the list of valid scores from the file
    private static ArrayList<Integer> readScores(String fileName) {
        ArrayList<Integer> parsedScores = new ArrayList<Integer>();
        try (BufferedReader reader = new BufferedReader(new FileReader(fileName))) {
            String score;
            while ((score = reader.readLine()) != null) {
                try {
                    // trim the line before parsing
                    Integer intScore = Integer.parseInt(score.trim());
                    if (intScore < 0 || intScore > 100) {
                        // strictly accept scores between 0 and 100

```

```

        // negative scores are not allowed
        // scores > 100 are not allowed
        invalidScores.add(score);
        continue;
    } else {
        parsedScores.add(intScore);
    }

    } catch (NumberFormatException e) {
        invalidScores.add(score);
    }
}
} catch (FileNotFoundException e) {
    System.out.println("Scores file was not found." + e.getMessage());
} catch (IOException e) {
    System.out.println("Could not read file: " + e.getMessage());
}
return parsedScores;
}

// method to return average of the list of scores,
// arguments : ArrayList of scores
// returns : double average, 0.0 if list is empty
public static double calculateAverage(ArrayList<Integer> scores) {
    if (scores.isEmpty()) {
        return 0.0;
    }
    Integer sum = 0;

    for (Integer score : scores) {
        sum += score;
    }
    return (double) sum / scores.size();
}

public static void analyzeScores ( List<Integer> scores ){
    for( Integer score: scores){
        if (score < minScore) {
            minScore = score;
        }
        if (score > maxScore) {
            maxScore = score;
        }
        String grade = getGrade(score);
        gradeCounter.merge(grade, 1, Integer::sum);
    }
}

// simple method to get grade for the score
private static String getGrade(int score) {

```

```

        if (score >= 90) {
            return "A";
        } else if (score >= 80) {
            return "B";
        } else if (score >= 70) {
            return "C";
        } else if (score >= 60) {
            return "D";
        } else {
            return "F";
        }
    }

    // Writes and prints the report
    public static void writeReport(List<Integer> scores,
        double avg, int high, int low,
        List<String> invalidScores,
        String outputFile) {
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(outputFile)))
        {
            writer.write("Total Scores Processed: ");
            writer.write(Integer.toString(scores.size()));
            writer.newLine();
            writer.write("Input Scores: ");
            writer.write(scores.toString());
            writer.newLine();
            writer.write("Invalid Lines Size: ");
            writer.write(Integer.toString(invalidScores.size()));
            writer.newLine();
            writer.write("Invalid Lines: ");
            writer.write(invalidScores.toString());
            writer.newLine();
            writer.newLine();
            writer.write("Average Score: ");
            writer.write(String.format("%.2f", avg));
            writer.newLine();
            writer.write("Highest Score: ");
            if (high >= 0) {
                writer.write(Integer.toString(high));
            } else {
                writer.write("N/A");
            }
            writer.newLine();
            writer.write("Lowest Score: ");
            if (low < 100) {
                writer.write(Integer.toString(low));
            } else {
                writer.write("N/A");
            }
            writer.newLine();
        }
    }

```

```

        writer.newLine();
        writer.write("Grade Distribution: ");
        writer.newLine();
        writer.write("A (90-100):      " + gradeCounter.get("A"));
        writer.newLine();
        writer.write("B (80-89):      " + gradeCounter.get("B"));
        writer.newLine();
        writer.write("C (70-79):      " + gradeCounter.get("C"));
        writer.newLine();
        writer.write("D (60-69):      " + gradeCounter.get("D"));
        writer.newLine();
        writer.write("F (below 60):    " + gradeCounter.get("F"));
        writer.newLine();
    } catch (IOException e) {
        System.out.println("Could not write file: " + e.getMessage());
    }
}
}

```